Polynomial Tables on the ENIAC

Charles Babbage's original motivation for the design of the Difference Engine was automating the computation of mathematical tables in the manner that they had been done manually. The technique used was known as the Method of Differences and took advantage of the characteristics of the derivatives of polynomial functions. It is very easy for the ENIAC to essentially simulate a Difference Engine in computing such tables. In an effort to tie together these two seminal parts of computer history, we examine here the process of implementing a program on the ENIAC to compute a table of values of a polynomial.

Method of Differences

The polynomial that we're going to work with in this example is $f(x) = x^3 - 20x^2 - 70x + 600$. Before we work out the values that we'll use to apply the Method of Differences, we note that the third derivative ($f^{(iii)}(x)$) is a constant value of 6. (We'll use that fact as a check on our initial values.)

Our first step in applying the Method of Differences is to manually create a table of the first few values of the function we want to tabulate. From that, we create another column which shows the differences between successive values of the function. That column is then followed by second differences between successive first differences, and so on until we get a column where the differences are a constant. This is where our small application of calculus comes into play. For a third-order polynomial the third derivative will be a constant, and the third-order difference will also be a constant and equal to the third-order derivative. This helps us to check our work. The following table shows the result of working this procedure out.

x	$f(x)$	Δ_1	Δ_2	Δ_3
0	600			
		-89		
1	511		-34	
		-123		6
2	388		-28	
		-151		6
3	237		-22	
		-173		6
4	64		-16	
		-189		
5	-125			

The result of this exercise is a set of initial values to be used in mechanizing the tabulation. Those values are the ones we find along the top diagonal row of this table, i.e. $\{600, -89, -34, 6\}$. With this as a starting point, we can then compute as many table entries as we wish with the following process:

Repeat for all table rows:

1. Record the current value of $f(x)$ as the next table entry.
2. Add the value of Δ_1 to that of $f(x)$.
3. Add the value of Δ_2 to that of Δ_1.
4. Add the value of Δ_3 to that of Δ_2.

For polynomials of different orders, the procedure will simply continue to update the non-constant differences.

Implementing on the ENIAC

Now we turn to how we implement this procedure on the ENIAC. Before we do anything else, we'll have to assign accumulators to each of the columns of our table. Because we want to include all the differences on the punched card output, we'll use the higher numbered accumulators. So we'll use Accumulator 17 for $f(x)$, 18 for Δ_1, 19 for Δ_2, and 20 for Δ_3.

With these assignments in mind, we will configure the printer (card punch). We'll enable printing of the low-order five digits of Accumulators 19 and 20 as well as all 10 digits of Accumulators 17 and 18. We'll also enable five digits of the master programmer for our values of x. The following lines set that up in the configuration file:

```
s pr.1 P
s pr.1-2 0
s pr.9 P
s pr.10 P
s pr.9-10 C
s pr.11 P
s pr.12 P
s pr.11-12 C
s pr.13 O
s pr.14 P
s pr.13-14 0
s pr.15 O
s pr.16 P
s pr.15-16 0
```

The next part of our program that we'll address is the initialization of the accumulators. Since we need four relatively small values, we'll use the switches on the constant transmitter panel 2 in groups of five digits.

```
s c.j1 P
s c.j10 0
s c.j9 0
s c.j8 6
s c.j7 0
s c.j6 0
s c.jr M
s c.j5 9
s c.j4 9
s c.j3 9
s c.j2 1
s c.j1 1
s c.k1 M
s c.k10 9
s c.k9 9
s c.k8 9
s c.k7 6
s c.k6 6
s c.kr P
s c.k5 0
s c.k4 0
s c.k3 0
s c.k2 0
s c.k1 6
```

For the negative numbers, we dial in the numbers in 10's complement form.

Now we'll set up the programming for the transfers from the constant transmitter to the accumulators. Note that with the high-order half of two of the constants are being transmitted, we will use shifter adapters to put them in the right place in the accumulators.

```
p c.o 1
p 1 ad.s.1.-5
p ad.s.1.-5 a17.beta
p 1 a18.alpha
p 1 ad.s.2.-5
p ad.s.2.-5 a19.beta
p 1 a20.alpha
p i.Io 1-1
p 1-1 c.25i
p 1-1 a17.1i
s c.s25 Jl
s a17.op1 beta
s a17.cc1 0
p c.25o 1-2
p 1-2 c.26i
p 1-2 a18.1i
s c.s26 Jr
s a18.op1 alpha
s a18.cc1 0
p c.26o 1-3
p 1-3 c.27i
p 1-3 a19.1i
s c.s27 Kl
s a19.op1 beta
s a19.cc1 0
p c.27o 1-4
p 1-4 c.28i
p 1-4 a20.1i
s c.s28 Kr
s a20.op1 alpha
s a20.cc1 0
p c.28o 1-5
```

Note that we are specifying the accumulator input terminals and operations by spelling out the Greek letters. The simulator allows this as well as the Unicode characters and the first five Latin letters.

Once the initialization is done, the first step of the repeated algorithm is punching a card.

```
p 1-5 i.Pi
p i.Po 1-6
```

After the card is punched, we engage the master programmer to control the repeating. Since we are using the master programmer decade counters for the value of x, the first step is to ensure that decades 14 and 18 are both assigned to stepper C.

```
s p.a14 C
s p.a18 C
```

Next we'll load the stage limits into the decade switches. We'll set stage 1 to 100 to set the limit on the rows of the table we're generating, and stage 2 to 99999. (Any non-zero value here would serve to prevent the master programmer from hitting the limit and advancing on beyond stage 2.)

```
s p.d18s1 0
s p.d17s1 0
```

```
s p.d16s1 1
s p.d15s1 0
s p.d14s1 0
s p.d18s2 9
s p.d17s2 9
s p.d16s2 9
s p.d15s2 9
s p.d14s2 9
```

Finally, we can instruct the master programmer that we'll need to complete two stages before resetting, and we'll plug the control lines for advancing the value of x and for triggering the addition cycle.

```
s p.cC 2
p 1-6 p.Ci
p p.C1o 1-7
```

The last part of the programming is the addition sequence where we add Accumulator 18 to 17, then 19 to 18, then 20 to 19. The final control pulse at the end of the sequence drives the print operation, forming the loop.

```
p a18.A 1
p a19.A 1
p a20.A 1
p 1 a17.alpha
p 1 a19.alpha
p 1-7 a17.2i
p 1-7 a18.5i
s a17.op2 alpha
s a17.cc2 0
s a18.op5 A
s a18.cc5 0
s a18.rp5 1
p a18.5o 1-8
p 1-8 a18.2i
p 1-8 a19.5i
s a18.op2 alpha
s a18.cc2 0
s a19.op5 A
s a19.cc5 0
s a19.rp5 1
p a19.5o 1-9
p 1-9 a19.2i
p 1-9 a20.5i
s a19.op2 alpha
s a19.cc2 0
s a20.op5 A
s a20.cc5 0
s a20.rp5 1
p a20.5o 1-5
```

When this is run, note that by the time the printer looks at the master programmer counter for $x = 100$, it will already be reset back to 0, so the last line will look like it's reporting $f(0)$ instead of $f(100)$.